

Codex Plan

发育特征 OT 方向的 P_{ij} 扩展计划

基因表达 $\times$ pseudotime $\times$ SR/potency $\times$ velocity

计划定位

本计划面向当前 `mignet_vertical_ablation` 代码库。目标是在已经存在的三维消融矩阵中继续扩展 P_{ij} 产生方式，使老师提出的四类 OT 信息来源——基因表达、发育 pseudotime、SR / signaling entropy / potency、velocity——都能作为可运行、可消融、可解释的 P_{ij} 方法接入当前 registry。Codex 阅读本计划后必须先暂停，等待用户确认后再开始修改代码。

目录

第一部分：当前状态与开发边界	3
1.1 当前已经实现了什么	3
1.2 已有基础设施	3
1.3 总体开发路线	4
第二部分：统一数学框架	5
2.1 所有 OT 方法最终都要输出同一种 P_{ij}	5
2.2 从 C_{ij} 到 transport plan T_{ij}	5
2.3 从 T_{ij} 到进入指标的 P_{ij}	5
第三部分：第一阶段 - developmental feature loader	7
3.1 为什么先做 loader	7
3.2 标准输入目录	7
3.3 CSV 格式	7
3.4 spot-level 到 domain-level 的聚合	8
3.5 需要新增的配置	8
3.6 第一阶段测试	8
第四部分：第二阶段 - pseudotime_ot 与 sr_ot	9
4.1 为什么第二阶段先做标量方法	9
4.2 pseudotime_ot	9
4.3 sr_ot	9
4.4 第二阶段实现要点	10
4.5 第二阶段测试	10
第五部分：第三阶段 - velocity_ot	11
5.1 为什么 velocity 要后做	11
5.2 velocity_ot 的数学定义	11
5.3 已有可复用代码	11

5.4 维度对齐原则	11
5.5 第三阶段测试	12
第六部分：第四阶段 - development_ot	13
6.1 综合方法的定位	13
6.2 development_ot 的代价函数	13
6.3 默认权重与消融	13
6.4 为什么不能只做综合方法	14
第七部分：代码修改清单与验收命令	15
7.1 config.py	15
7.2 registry.py	15
7.3 CLI	15
7.4 输出诊断增强	16
7.5 验收命令	16
7.6 推荐开发顺序	17
第八部分：科学解释口径	18
8.1 新增方法的科学含义	18
8.2 最终要回答的问题	18

第一部分：当前状态与开发边界

1.1 当前已经实现了什么

从当前源码看,OT 类方法已经有两条完整实现: `expr_ot` 和 `energy_entropy_ot`。它们都已经进入 `PIJ_METHODS` 和 `PIJ_METHOD_REGISTRY`, 因此可以直接进入现有消融矩阵。老师提出的 `pseudotime`、`SR` 和 `velocity` 方向还没有成为可运行的 Pij method。

老师提出的方向	当前状态	代码结论
基因表达 / 表达特征 OT	已实现	对应 <code>mignet_ce/pij/expr_ot.py</code> , 已注册。
Energy / entropy OT	已实现第一版	对应 <code>energy_entropy_ot.py</code> , 使用 <code>expression</code> 、 <code>spatial</code> 、 <code>graph energy cost</code> 。
<code>pseudotime</code> OT	未实现	没有 <code>pseudotime_ot.py</code> , 也没有 <code>developmental feature loader</code> 。
<code>SR / potency</code> OT	未实现	没有 <code>sr_ot.py</code> , 也没有读取 <code>sr</code> 或 <code>potency_score</code> 的接口。
<code>velocity</code> OT	半成品基础	<code>cost_components.py</code> 已有 <code>velocity direction cost</code> , 但缺少 <code>velocity_ot.py</code> 。
<code>development</code> OT	未实现	需要等单因素方法跑通后, 再实现综合方法。

1.2 已有基础设施

当前代码已经具备新增 OT 方法的底座:

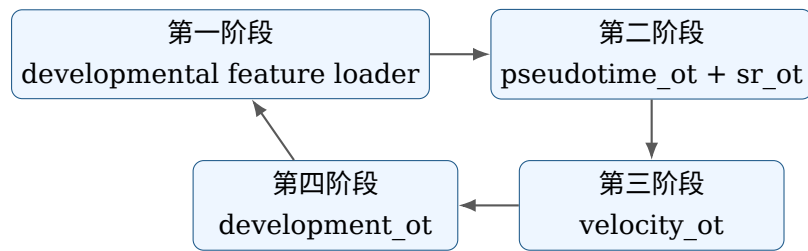
- `mignet_ce/transition/ot.py` 已有 `build_entropic_ot_kernel(...)`, 支持 POT 优先和内部 Sinkhorn fallback。
- `mignet_ce/transition/cost_components.py` 已有 `expression`、`spatial`、`scalar`、`velocity direction` 等 `cost` 函数。
- `mignet_ce/pij/_ot_common.py` 已有 `run_ot_pij_method(...)`, 可以作为所有新增 OT Pij 方法的模板。
- `TransitionKernels` 已经可以承接 `precomputed kernel`, 因此新增方法无需重写 EI / DI / TE 计算逻辑。

开发边界

新增方法必须接入当前 `config.py`、`registry.py` 和 `run_mignet_vertical_ablation.py` 体系, 不能另写孤立 pipeline。可以参考本地 `reference` 项目, 但新代码运行时不能 `import` 或读取本地 `reference` 文件夹。外部发育特征应通过标准 CSV 输入, 而不是硬绑定某个大型外部工具。

1.3 总体开发路线

本计划把开发拆成四个阶段：



第一阶段先解决数据入口；第二阶段实现两个最稳的标量发育特征 OT；第三阶段实现向量型 velocity OT；第四阶段再把所有信息合成 development OT。这个顺序可以避免一开始就做过重、过综合的方法，导致结果不可解释。

第二部分：统一数学框架

2.1 所有 OT 方法最终都要输出同一种 P_{ij}

对一个时间跨度 $t_0 \rightarrow t_1$ ，时间点 t_0 是 source time，时间点 t_1 是 target time。设 source 有 n 个 units，target 有 m 个 units。第 i 个 source unit 到第 j 个 target unit 的匹配代价记为 C_{ij} 。

代价矩阵的含义

C_{ij} = source unit i 转移到 target unit j 的代价。

C_{ij} 越小，表示 i 越适合对应到 j 。不同 Pij 方法的主要差别，不是 OT 求解器不同，而是 C_{ij} 怎么构造。例如 `expr_ot` 只用表达 / 网络特征距离，`pseudotime_ot` 额外加入发育进度，`velocity_ot` 额外加入动态方向。

2.2 从 C_{ij} 到 transport plan T_{ij}

给定价矩阵后，代码通过熵正则 OT 求 transport plan：

熵正则 OT

$$T^* = \arg \min_T \langle T, C \rangle + \varepsilon \sum_{i,j} T_{ij} \log T_{ij}.$$

这里 T_{ij} 表示从 source unit i 分配到 target unit j 的运输质量； $\langle T, C \rangle$ 是所有运输质量乘以对应代价后的总成本； ε 控制平滑程度。 ε 越小，匹配越尖锐； ε 越大，匹配越平滑。

在 balanced OT 中，source mass 和 target mass 默认均匀：

$$a_i = \frac{1}{n}, \quad b_j = \frac{1}{m}.$$

Sinkhorn 过程可以理解为先由代价生成 kernel：

$$K_{ij} = \exp \left(-\frac{C_{ij}}{\varepsilon} \right),$$

再通过迭代得到缩放因子 u_i 和 v_j ：

$$T_{ij} = u_i K_{ij} v_j.$$

u_i 和 v_j 的作用是让 transport plan 满足 source 与 target 的质量约束。

2.3 从 T_{ij} 到进入指标的 P_{ij}

现有 EI / DI / TE 指标需要的是 row-stochastic transition kernel，所以 transport plan 还需要按行归一化：

真正进入指标的转移概率

$$P_{ij} = \frac{T_{ij}}{\sum_{k=1}^m T_{ik}}, \quad \sum_{j=1}^m P_{ij} = 1.$$

这里 P_{ij} 表示：给定 source unit i ，它转移或对应到 target unit j 的条件概率。新增方法必须把 lower layer 和 upper layer 的 P 分别写入：

```
kernels.p_lower[(t0, t1)] = lower_layer_P  
kernels.p_upper[(t0, t1)] = upper_layer_P
```

这样 vertical pipeline 就可以继续复用原有 EI / DI / TE 计算。

第三部分：第一阶段 - developmental feature loader

3.1 为什么先做 loader

pseudotime、SR、potency 和 velocity 都不是当前 `build_graph_feature_result(...)` 自动产生的变量。它们更像外部发育特征，来源可能是 Scanpy DPT、SCENT / LandSCENT、CytoTRACE2、scVelo、CellRank 或用户预先计算的 CSV。因此，第一阶段必须先做统一读取层，而不是直接写多个 Pij 方法。

新增文件

```
mignet_ce/io/developmental_features.py
```

3.2 标准输入目录

建议统一放在 data root 下：

```
$data_root/developmental_features/  
spot/  
  heart_11.5_features.csv  
  heart_12.5_features.csv  
louvain_less_than5/  
  heart_11.5_features.csv  
louvain_k150/  
  heart_11.5_features.csv  
louvain_k40/  
  heart_11.5_features.csv  
seurat_less_than5/  
seurat_k150/  
seurat_k40/  
spatial_domain_less_than5/  
spatial_domain_k150/  
spatial_domain_k40/
```

文件名使用 `organ_stage_features.csv`，例如 `heart_11.5_features.csv`。

3.3 CSV 格式

最小格式：

```
unit_id,pseudotime,sr,potency_score  
unit_001,0.12,0.87,0.91  
unit_002,0.31,0.76,0.78
```

包含 velocity 时增加若干列：


```
unit_id,pseudotime,sr,potency_score,velocity_0,velocity_1,velocity_2
unit_001,0.12,0.87,0.91,0.03,-0.01,0.22
unit_002,0.31,0.76,0.78,0.08,0.02,0.10
```

3.4 spot-level 到 domain-level 的聚合

如果用户只提供 spot-level feature, 但当前层级是 domain-level, loader 应该使用 spot-domain 映射表聚合。对于 domain D , pseudotime 聚合为:

domain-level pseudotime 聚合

$$pt_D = \frac{1}{|D|} \sum_{s \in D} pt_s.$$

这里 s 表示属于 domain D 的 spot, $|D|$ 是 domain 内 spot 数。SR、potency 和 velocity 每个维度也按同样方式聚合。默认使用 mean, 可配置为 median。

3.5 需要新增的配置

```
development_feature_root: Path | None = None
pij_feature_aggregation: str = "mean"      # mean / median
pij_missing_feature_policy: str = "error"  # error / impute_mean / ignore
pij_pseudotime_weight: float = 0.5
pij_sr_weight: float = 0.5
pij_potency_weight: float = 0.5
pij_velocity_weight: float = 0.5
pij_backward_pseudotime_weight: float = 0.0
pij_reverse_potency_weight: float = 0.0
```

validate() 必须检查权重非负、aggregation / missing policy 合法, 并在 developmental 方法缺少 feature root 时清晰报错。

3.6 第一阶段测试

新增 tests/test_developmental_features.py, 至少覆盖:

- 读取 unit-level CSV;
- 读取 spot-level CSV 并聚合到 domain-level;
- 缺失 unit 时 error 模式报错;
- impute_mean 模式可运行并写 metadata warning;
- velocity_* 列能正确读成二维矩阵。

第四部分：第二阶段 - pseudotime_ot 与 sr_ot

4.1 为什么第二阶段先做标量方法

pseudotime 和 SR 都是标量。它们的实现难度低于 velocity，也更容易解释。因此第二阶段先实现这两个方法，用来回答：在表达相似性之外，加入发育进度或发育潜能是否改变 EI_gain 结论。

4.2 pseudotime_ot

新增文件

mignet_ce/pij/pseudotime_ot.py

核心代价：

pseudotime OT cost

$$C_{ij} = w_{expr} C_{ij}^{expr} + w_{pt} |pt_i - pt_j| + w_{back} \max(0, pt_i - pt_j).$$

这里 C_{ij}^{expr} 是已有 graph feature distance; pt_i 是 source unit i 的 pseudotime; pt_j 是 target unit j 的 pseudotime; $|pt_i - pt_j|$ 惩罚发育阶段差异太大的匹配; $\max(0, pt_i - pt_j)$ 是 backward penalty, 惩罚从更晚 pseudotime 倒流到更早 pseudotime。

第一版建议默认：

```

pij_expr_weight = 1.0
pij_pseudotime_weight = 0.5
pij_backward_pseudotime_weight = 0.0

```

不默认开启 backward penalty，是因为不同器官和不同阶段不一定满足严格 pseudotime 单调推进。

4.3 sr_ot

新增文件

mignet_ce/pij/sr_ot.py

核心代价：

SR / potency OT cost

$$C_{ij} = w_{expr} C_{ij}^{expr} + w_{sr} |SR_i - SR_j|.$$

这里 SR_i 是 source unit 的 signaling entropy rate、分化潜能或用户提供的 potency-like score； SR_j 是 target unit 的对应值； $|SR_i - SR_j|$ 表示两个 unit 的发育潜能状态差异。

如需加入潜能下降假设，可加入：

$$C_{ij} \leftarrow C_{ij} + w_{rev} \max(0, SR_j - SR_i).$$

但第一版默认 `pij_reverse_potency_weight = 0.0`，避免过早强加方向性假设。

4.4 第二阶段实现要点

- 两个方法都复用 `pairwise_scalar_cost(...)` 构造标量距离矩阵。
- 两个方法都复用 `run_ot_pij_method(...)`，不要重写 Sinkhorn。
- metadata 中必须记录使用了哪些 cost components、权重、是否存在缺失 feature。
- 缺少 pseudotime 或 sr 列时，必须抛出清晰错误。

4.5 第二阶段测试

新增或扩展 `tests/test_developmental_pij_methods.py`：

- `pseudotime_ot` 在提供 pseudotime CSV 时能输出 lower / upper kernels；
- `sr_ot` 在提供 sr CSV 时能输出 lower / upper kernels；
- 输出 shape 正确、非负、finite、行和为 1；
- 缺少必要列时报错信息清晰。

第五部分：第三阶段 - velocity_ot

5.1 为什么 velocity 要后做

velocity 是向量，不是标量。它需要保证 velocity vector 与 graph feature representation 处于同一个维度空间，否则方向余弦没有意义。因此 velocity_ot 比 pseudotime_ot 和 sr_ot 更容易出错，应放在第三阶段。

5.2 velocity_ot 的数学定义

新增文件

mignet_ce/pij/velocity_ot.py

velocity cost 写成：

velocity direction cost

$$C_{ij}^{vel} = 1 - \cos(v_i, z_j - z_i).$$

这里 v_i 是 source unit i 的 velocity vector； z_i 是 source unit 的 graph feature； z_j 是 target unit 的 graph feature； $z_j - z_i$ 表示从 source 状态走向 target 状态的方向。如果 v_i 与 $z_j - z_i$ 方向一致，则 cost 低；如果方向相反，则 cost 高。

完整代价：

velocity OT 综合 cost

$$C_{ij} = w_{expr} C_{ij}^{expr} + w_{vel} C_{ij}^{vel}.$$

5.3 已有可复用代码

当前 cost_components.py 已有：

`pairwise_velocity_direction_cost(source_features, target_features, source_velocity)`

第三阶段主要任务不是写新公式，而是把 feature loader、method class、registry 和 CLI 打通。

5.4 维度对齐原则

第一版要求 velocity 维度必须等于 graph feature 维度。如果不一致，直接报错：

`velocity_ot requires velocity_* dimension to match graph feature dimension.`

不要在第一版里自动 PCA、自动投影或自动补维。自动投影会使 velocity 的生物含义变得不清楚。

5.5 第三阶段测试

- velocity 维度与 feature 维度一致时可运行；
- velocity 维度不一致时报错；
- 缺少 velocity_* 列时报错；
- 输出 kernel shape 正确、非负、finite、行和为 1；
- metadata 中记录 velocity cost 的 weight、min、max、mean。

第六部分：第四阶段 - development_ot

6.1 综合方法的定位

development_ot 不应该先于单因素方法实现。它应当在 expr_ot、pseudotime_ot、sr_ot 和 velocity_ot 都可单独运行之后，再作为综合方法加入。

它回答的问题是：如果把表达相似性、空间邻近、发育进度、发育潜能和动态方向全部合成一个 OT cost，跨时间 Pij 是否比任一单因素方法更能支持 EI_gain?

6.2 development_ot 的代价函数

新增文件

mignet_ce/pij/development_ot.py

综合代价：

development OT cost

$$C_{ij} = w_{expr}C_{ij}^{expr} + w_{spatial}C_{ij}^{spatial} + w_{pt}C_{ij}^{pseudotime} + w_{sr}C_{ij}^{SR} + w_{vel}C_{ij}^{velocity}.$$

各项含义如下：

cost 组件	含义
C_{ij}^{expr}	source 与 target 的 graph feature / expression 状态差异。
$C_{ij}^{spatial}$	source 与 target 的空间坐标距离。
$C_{ij}^{pseudotime}$	source 与 target 的发育进度差异。
C_{ij}^{SR}	source 与 target 的 SR / potency 状态差异。
$C_{ij}^{velocity}$	source velocity 是否指向 target 状态。

6.3 默认权重与消融

第一版建议默认：

```

pij_expr_weight = 1.0
pij_spatial_weight = 0.2
pij_pseudotime_weight = 0.5
pij_sr_weight = 0.5
pij_velocity_weight = 0.5

```

这些只是初始权重。正式实验建议做权重消融：

development_no_velocity
development_no_sr
development_no_pseudotime
development_all

6.4 为什么不能只做综合方法

如果只做 development_ot, 结果很难解释。假设它的 EI_gain 很高, 我们无法判断是 pseudotime、SR、velocity 还是 spatial cost 起作用。因此必须保留单因素方法:

方法	它回答的问题
expr_ot	只看表达 / 网络状态相似, 是否足够?
pseudotime_ot	加入发育进度以后, EI_gain 是否更稳定?
sr_ot	加入分化潜能或 signaling entropy 后, 是否更符合发育涌现?
velocity_ot	加入动态方向后, Pij 是否更像真实发育转移?
development_ot	综合发育信息是否优于任一单因素?

第七部分：代码修改清单与验收命令

7.1 config.py

扩展 PIJ_METHODS:

```
PIJ_METHODS = {
    "joint_nmf", "laplacian", "3dot", "slat",
    "expr_ot", "energy_entropy_ot",
    "pseudotime_ot", "sr_ot", "velocity_ot", "development_ot",
}
```

建议扩展 presets:

```
PIJ_METHOD_PRESETS = {
    "core": ("joint_nmf", "laplacian", "3dot", "slat"),
    "ot_basic": ("expr_ot", "energy_entropy_ot"),
    "development_scalar": ("pseudotime_ot", "sr_ot"),
    "development_all": ("pseudotime_ot", "sr_ot", "velocity_ot", "development_ot"),
    "all": (... 全部方法...),
}
```

7.2 registry.py

新增 imports:

```
from mignet_ce.pij.pseudotime_ot import PseudotimeOTPijMethod
from mignet_ce.pij.sr_ot import SROTPijMethod
from mignet_ce.pij.velocity_ot import VelocityOTPijMethod
from mignet_ce.pij.development_ot import DevelopmentOTPijMethod
```

新增 registry 项:

```
"pseudotime_ot": PseudotimeOTPijMethod,
"sr_ot": SROTPijMethod,
"velocity_ot": VelocityOTPijMethod,
"development_ot": DevelopmentOTPijMethod,
```

7.3 CLI

scripts/run_mignet_vertical_ablation.py 需要透传:

```
--development-feature-root
--pij-feature-aggregation
--pij-missing-feature-policy
--pij-pseudotime-weight
```



```
--pij-sr-weight
--pij-potency-weight
--pij-velocity-weight
--pij-backward-pseudotime-weight
--pij-reverse-potency-weight
```

7.4 输出诊断增强

现有 `export_pij=True` 已经输出 npz 文件和 top-k CSV; developmental OT 方法建议在 kernel metadata 中额外写入:

```
cost_components
cost_summary
missing_feature_policy
feature_columns_used
feature_aggregation
```

如果增强 top-k CSV, 建议列包括:

```
source_unit,target_unit,probability,rank,time_pair,space,pij_method,
expression_cost,spatial_cost,pseudotime_cost,
sr_cost,velocity_cost,main_cost
```

这样后续可以解释某条 P_{ij} 边到底是因为表达相似、发育阶段接近、SR 接近, 还是 velocity 指向。

7.5 验收命令

基础测试:

```
pytest -q
```

只跑 pseudotime 与 SR:

```
python scripts/run_mignet_vertical_ablation.py \
--data-root "/home/jovyan/public/datasets/Mouse-embryo/E1S1_domain_factory" \
--development-feature-root
↪ "/home/jovyan/public/datasets/Mouse-embryo/E1S1_domain_factory/developmental_features" \
--output-root "/home/jovyan/work/2026 Causality/output/mignet_vertical_ablation_development_scalar" \
--network-methods legacy_mixed_grn_cci cross_cell_multilayer \
--pij-methods pseudotime_ot sr_ot \
--pair-preset spatial_domain_all \
--organs heart brain lung \
--time-points 11.5 12.5 13.5 14.5 \
--export-pij
```

单独跑 velocity:

```
python scripts/run_mignet_vertical_ablation.py \
--data-root "/home/jovyan/public/datasets/Mouse-embryo/E1S1_domain_factory" \
--development-feature-root
↪ "/home/jovyan/public/datasets/Mouse-embryo/E1S1_domain_factory/developmental_features" \
--output-root "/home/jovyan/work/2026 Causality/output/mignet_vertical_ablation_velocity" \
--network-methods legacy_mixed_grn_cci \
--pij-methods velocity_ot \
--pair-preset spatial_domain_all \
--organs heart brain lung \
--time-points 11.5 12.5 13.5 14.5 \
--export-pij
```

7.6 推荐开发顺序

1. 第一轮: 实现 `developmental_features.py`、`pseudotime_ot.py`、`sr_ot.py`、`config / registry / CLI / tests`。
2. 跑 `pytest -q` 和一个小型 smoke test, 然后暂停等待用户确认。
3. 第二轮: 实现 `velocity_ot.py` 和 `velocity` 相关测试。
4. 第三轮: 实现 `development_ot.py`, 增强 top-k cost diagnostics, 并跑 full ablation smoke test。

第八部分：科学解释口径

8.1 新增方法的科学含义

新增方法不是简单增加相似度函数，而是把 P_{ij} 背后的生物学假设拆开：

Pij 方法	P_{ij} 的解释
expr_ot	表达 / 网络状态相似的 unit 更可能跨时间对应。
pseudotime_ot	表达相似且发育进度合理的 unit 更可能对应。
sr_ot	表达相似且分化潜能 / signaling entropy 接近的 unit 更可能对应。
velocity_ot	source unit 的动态变化方向指向 target unit 时，对应概率更高。
development_ot	综合表达、空间、pseudotime、SR、velocity 的发育过程对应。

8.2 最终要回答的问题

完成这些方法后，消融矩阵就不只是比较“哪个 Pij 数值高”，而是回答：

核心科学问题

在不同生物学假设下，跨时间 P_{ij} 的改变是否会改变 EI_gain、DI 和 TE？如果结论稳定，说明因果涌现信号不依赖某一个特定 Pij 构造；如果结论只在某类发育约束 OT 下出现，说明因果涌现更可能依赖发育进程方向。